

Quadtree Report - CS163 - G58

1. Introduction

2. Definition:

3. Quadtree Implementation

Implement quad tree

3.1 A simple 2D point

3.2 A **Rectangle** space, including the following methods:

3.3 Quad Tree

4. Quadtree - Visualizer Demo:

5. Time complex, space complex:

Time Complexity of Quadtree

Space Complexity of Quadtree

6. Application of Quadtree:

6.1 Range Query:

6.2. Spherical Region Query

6.2. Point Location

How Quadtree Works in the SDK:

Initializing the Quadtree

Adding Points to the Quadtree

Searching for Points in the Quadtree

Finding Nearby Restaurants Near the University of Science (Ho Chi Minh City)

Additional Applications:

7. Program assignment

Example 1

Problem

Analysis

Setting

Example 2

Problem

Sample input

Sample output

Setting

8. Conclusion:

References:

1. Introduction

When you search for “Best restaurants near me” on Google Maps, it quickly displays a list of good restaurants within a 2km radius. With billions of restaurants worldwide, the system must process a vast amount of spatial data. Finding a restaurant within a specific area requires fast and efficient query processing. This application is part of a **Geographic Information System (GIS)**. How does Google Maps achieve this? This article analyzes the Quadtree data structure: Definition, Concept, Implementation, Visualization Demo, Time Complexity, Space Complexity, Types of Quadtree, and some applications of Quadtree.

2. Definition:

Quadtree is a tree data structure invented by [Raphael Finkel](#) and [J.L. Bentley](#) in 1974.

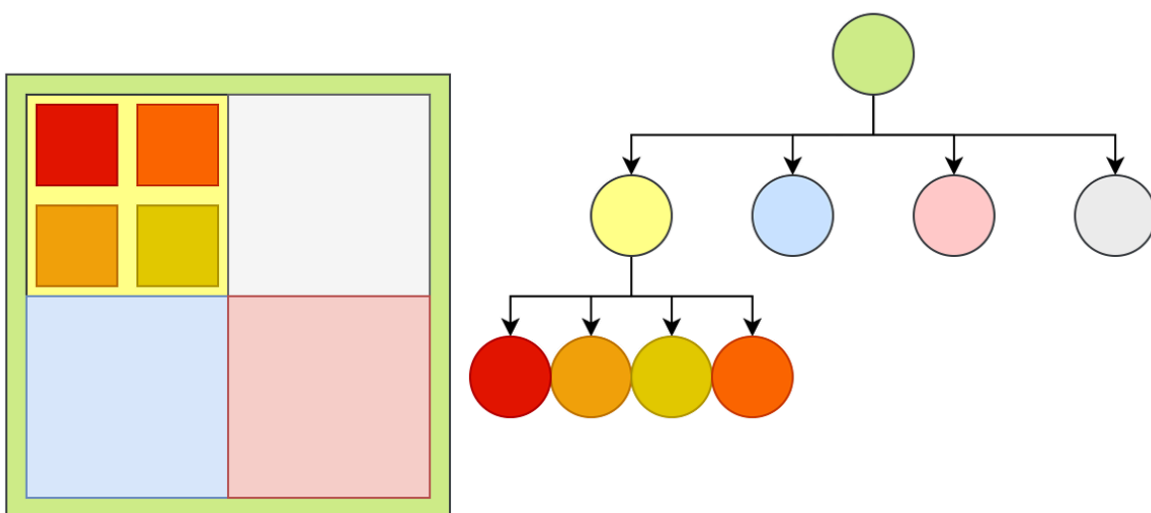
The Quadtree is a

hierarchical spatial data structure. It is a tree where each level represents a further refinement of the considered space. Although there are many types of Quadtrees and they can be generalized to any dimension, the fundamental idea is always a **recursive decomposition of space**, allowing us to store only essential or interesting information about the space.

More details:

[Quadtree Paper](#)

In this article, we focus on **Quadtree in 2D space**. Typically, 2D space is considered a square grid with unit squares in a two-dimensional Euclidean plane.



More references: [Quadtree Notes](#)

To understand it better, **Quadtrees** are trees used to efficiently store point data in a **2D space**. Each node has at most four children. We can construct a **Quadtree** from a two-dimensional area using the following steps:

1. Divide the current two-dimensional space into four equal regions.
2. If a region contains one or more points, create a child node, storing in it the corresponding sub-region.
3. If a region does not contain any points, do not create a child node.

3. Quadtree Implementation

We are now ready to define the structure of a **Quad Tree**:

- A **Quad Tree** is a **tree data structure**, specifically a hierarchical spatial partitioning structure where each node has at most **four** children.
- Each node manages a **rectangular region** of space and stores a set of points within that region.
- The root node covers the entire space from **(1,1) to (N,M)**.
- Each node has a **capacity** (the maximum number of points it can store before subdividing).
 - If the number of points in the region is $\leq$ **capacity**, the node simply stores the points without further subdivision.
 - If the number of points in the region **exceeds capacity**, the node is **subdivided into four equal quadrants** (by splitting the region in half along both dimensions), and the points are redistributed among the child nodes.
- Once a node is subdivided, it no longer directly stores points; instead, the points are pushed down into the appropriate child nodes.
- The **height of the tree** depends on the spatial distribution of points. In the worst case, where points are evenly distributed across the entire space, the height can be **$O(\log N + \log M)$** since each subdivision reduces the region size along both dimensions.

Implement quad tree

3.1 A simple 2D point

```

struct point {
    double x, y;
    point(double _x, double _y) : x(_x), y(_y) {}
};

```

3.2 A Rectangle space, including the following methods:

```

struct rect {
    point TopLeft, BottomRight;
    rect(point TL, point BR) : TopLeft(TL), BottomRight(BR) {}
    bool containsPoint(point p); // Checks whether a given point exists within the space
    bool intersects(rect r); // Checks whether two spaces overlap
};

bool rect::containsPoint(point p) {
    return (p.x >= TopLeft.x && p.x <= BottomRight.x && p.y >= TopLeft.y && p.y <= BottomRight.y);
}

bool rect::intersects(rect r) {
    return !(r.TopLeft.x > BottomRight.x || r.BottomRight.x < TopLeft.x || r.TopLeft.y > BottomRight.y || r.BottomRight.y < TopLeft.y);
}

```

3.3 Quad Tree

```

#include "rect.h"
struct QuadTree {
    rect boundary; // Boundaries of the QuadTree
    int capacity; // Max points before subdivision
    std::vector<point> points; // Points in this QuadTree node
    QuadTree *NE, *NW, *SE, *SW; // Children nodes
    QuadTree(rect _boundary, int _capacity) : boundary(_boundary), capacity(_capacity), NE(NULL), NW(NULL), SE(NULL), SW(NULL) {}
    bool insert(point p); // Insert a point
    void subdivide(); // Subdivide into 4 children
    std::vector<point> queryRange(rect r); // Find points within a range
}

```

```
void clear(); // Clear the QuadTree
};
```

3.3.1 Subdivide() to split quadtree into 4.

```
void QuadTree::subdivide() {
    point TL = boundary.TopLeft;
    point BR = boundary.BottomRight;
    point center = point((TL.x + BR.x) / 2, (TL.y + BR.y) / 2);
    NE = new QuadTree(rect(point(center.x, TL.y), point(BR.x, center.y)), capacity);
    NW = new QuadTree(rect(TL, center), capacity);
    SE = new QuadTree(rect(point(center.x, center.y), BR), capacity);
    SW = new QuadTree(rect(point(TL.x, center.y), point(center.x, BR.y)), capacity);
}
```

3.3.2 Insert(Point) to add a point to the quadtree

```
bool QuadTree::insert(point p) {
    if (!boundary.containsPoint(p)) {
        return false; // Point outside boundary
    }
    if (points.size() < capacity && NE == NULL) {
        points.push_back(p); // Add point if there's space
        return true;
    }
    if (NE == NULL) {
        subdivide(); // Subdivide if necessary
    }
    // We have to add the points/data contained in this quad array to the new
    // quads if we only want the last node to hold the data
    for (vector<point>::iterator it = points.begin(); it != points.end(); ++it) {
        if (NE->insert(*it)) continue;
        if (NW->insert(*it)) continue;
        if (SE->insert(*it)) continue;
        if (SW->insert(*it)) continue;
    }
}
```

```

points.clear();

if (NE→insert(p)) return true;
if (NW→insert(p)) return true;
if (SE→insert(p)) return true;
if (SW→insert(p)) return true;

return false; // Should never happen
}

```

3.3.3 Query(Range) returns the points within the search area.

```

vector<point> QuadTree::queryRange(rect r) {
    vector<point> pointsInRange;
    if (!boundary.intersects(r)) return pointsInRange; // No intersection
    for (vector<point>::iterator it = points.begin(); it != points.end(); ++it) {
        if (r.containsPoint(*it)) {
            pointsInRange.push_back(*it);
        }
    }
    if (NE == NULL) return pointsInRange; // No children
    vector<point> NEPoints = NE→queryRange(r);
    vector<point> NWPoints = NW→queryRange(r);
    vector<point> SEPoints = SE→queryRange(r);
    vector<point> SWPoints = SW→queryRange(r);
    pointsInRange.insert(pointsInRange.end(), NEPoints.begin(), NEPoints.end());
    pointsInRange.insert(pointsInRange.end(), NWPoints.begin(), NWPoints.end());
    pointsInRange.insert(pointsInRange.end(), SEPoints.begin(), SEPoints.end());
    pointsInRange.insert(pointsInRange.end(), SWPoints.begin(), SWPoints.end());
    return pointsInRange;
}

```

3.3.4 Clear quad tree

```

void QuadTree::clear() {
    points.clear();
    if (NE != NULL) {
        NE→clear();
        NW→clear();
        SE→clear();
        SW→clear();
    }
    delete NE;
    delete NW;
    delete SE;
    delete SW;
    NE = NULL;
    NW = NULL;
    SE = NULL;
    SW = NULL;
}

```

4. Quadtree - Visualizer Demo:

I used OpenDSA's Visualizer: [OpenDSA Quadtree](#) to visualize the Quadtree. Since the process is relatively long, I recorded a video for easier viewing: [\[Quad Tree Step By Step Demo\]](#).

5. Time complex, space complex:

Time Complexity of Quadtree

Constructing a quadtree involves inserting N points, where each insertion typically takes $O(\log N)$ time. Consequently, the overall time complexity for building a quadtree is $O(N \log N)$.

There are multiple ways to construct a quadtree. If it is built from a uniform grid (e.g., a matrix of pixels), the tree height is $O(\log N)$. However, if points are inserted sequentially—similar to a Binary Search Tree (BST)—the worst-case scenario occurs when all points are sorted along one axis. In this case, the tree degenerates into a linked list with a height of $O(N)$.

For example, inserting points sequentially as follows:

1. Insert (0,0)
2. Insert (1,1)
3. Insert (2,2)
4. Insert (3,3)
5. ...
6. Insert (N-1,N-1)

Each point is placed in the upper-right quadrant of the previous node, resulting in a structure where each node has exactly one child. This leads to a worst-case tree height of $O(N)$, making operations such as insertion and search take $O(N)$ time in the worst case.

Thus, the efficiency of a quadtree depends on the insertion order and construction method, highlighting the importance of balancing strategies to maintain logarithmic depth.

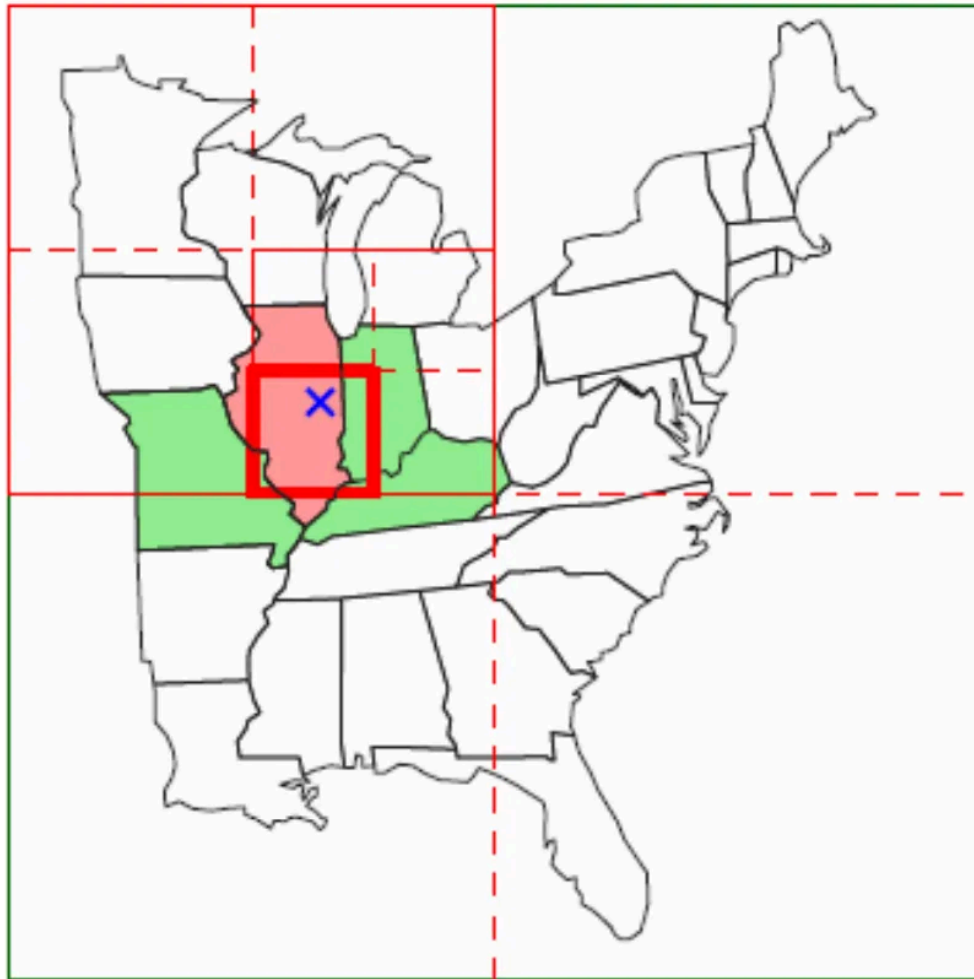
Space Complexity of Quadtree

$O(k \log_4 N * M) = O(k \log_2 N)$ where the size of the region is NxM ($N \geq M$).

Assume the quadtree manages k points.

6. Application of Quadtree:

One notable application is **Geographic Information Systems (GIS)**. Quadtree-based **spatial indexing** allows efficient storage and retrieval of map data, enabling applications such as geocoding, routing, and spatial analysis.



More references: [Spatial Indexing with Quadrees](#)

6.1 Range Query:

Given a PR quadtree and a query region R (which is possibly open-ended), the result of a range query is all of the stored points in the PR quadtree that lie within R . This operation may correspond to finding records in a database that meet a certain criteria. To do the range query, first we find the smallest node/cell u in the tree that completely contains R . For each child cell v of u :

- If v lies completely within R , add the points of v 's subtree to the query result and continue the query.
- If v and R do not intersect, discard v and continue with the other children.
- If v and R intersect, process the children of v similarly in turn (brute-force check leaves).

<https://people.scs.carleton.ca/~maheshwa/courses/5703COMP/16Fall/quadrees-paper.pdf>

When the user zooms in on an area, Maps uses a range query to display only data points (restaurants, stores, etc.) within the visible region, minimizing data load and increasing

rendering speed.

<https://github.com/Flokey82/geoquad?tab=readme-ov-file>

6.2.Spherical Region Query

Given a PR quadtree, a query point q , and a distance $r > 0$, the result of a spherical region query is all of the stored points in the tree that lie in the circle centred at q with radius r . An example query may be a request for all hospitals within 10 km of someone's current location. The query is answered by walking down the tree from the root to the cell containing q while exploring and ruling out cells on the way. For each cell v we have not yet ruled out:

- If the farthest spatial point in v is less than r from q , add all stored points in v to the query result, remove v from consideration, and continue the query.
- If the closest spatial point in v is more than r from q , remove v from consideration, and continue the query.
- Otherwise, remove v and add its children for consideration (brute-force check the data points of a leaf's cell).

Maps uses a spherical region query to provide services such as nearby location searches (e.g., restaurants, gas stations, hospitals) within a specific radius from the user's location.

6.2.Point Location

Given a quadtree for a polygonal map and a query point q , the result of a point location query is the polygon in which q lies. An example of such a query is a traveller asking a map service which city they are currently in. The quadtree we will consider here is a slight variation of the ones seen above. The difference is that the stopping criterion for the subdivision is when a cell is intersected by a constant number of polygons. With such a quadtree, the point location query is rather simple: find the leaf in the quadtree that contains q , then check which of the $O(1)$ polygons contain q (in other words, brute-force check against the polygons in the leaf of q).

Maps uses a point location query to determine the user's exact position on the map, such as identifying the city, district, or country the user is in.

How Quadtree Works in the SDK:

The quadtree is integrated into the SDK's mapping library for iOS. Google's developer documentation has made its implementation open-source.

<https://developers.google.com/maps/documentation/ios-sdk/utility/quadtree?hl=en>

Initializing the Quadtree

A quadtree is initialized with a defined spatial boundary (bounding box). In the provided example, the bounding box is defined by the coordinates `(minX, minY)` and `(maxX, maxY)` :

```
let bounds = GQTBounds(minX: -2, minY: -2, maxX: 2, maxY: 2)
guard let tree = GQTPointQuadTree(bounds: bounds) else {
    return
}
```

- **GQTBounds**: Represents a rectangular bounding box in 2D space.
- **GQTPointQuadTree**: A class representing a quadtree, initialized with a defined spatial boundary.

Adding Points to the Quadtree

Data points are inserted into the quadtree using the `add` method. Each point is represented by an object conforming to the `GQTPointQuadTreeItem` protocol:

```
tree.add(QuadTreeItem(point: GQTPoint(x: -1, y: -1)))
tree.add(QuadTreeItem(point: GQTPoint(x: -1, y: 1)))
tree.add(QuadTreeItem(point: GQTPoint(x: 1, y: 1)))
tree.add(QuadTreeItem(point: GQTPoint(x: 1, y: -1)))
```

- **GQTPoint**: Represents a point in 2D space with coordinates `(x, y)` .
- **QuadTreeItem**: A custom class conforming to the `GQTPointQuadTreeItem` protocol, representing an item in the quadtree.

Searching for Points in the Quadtree

To search for points within a specific spatial region, the `search` method is used:

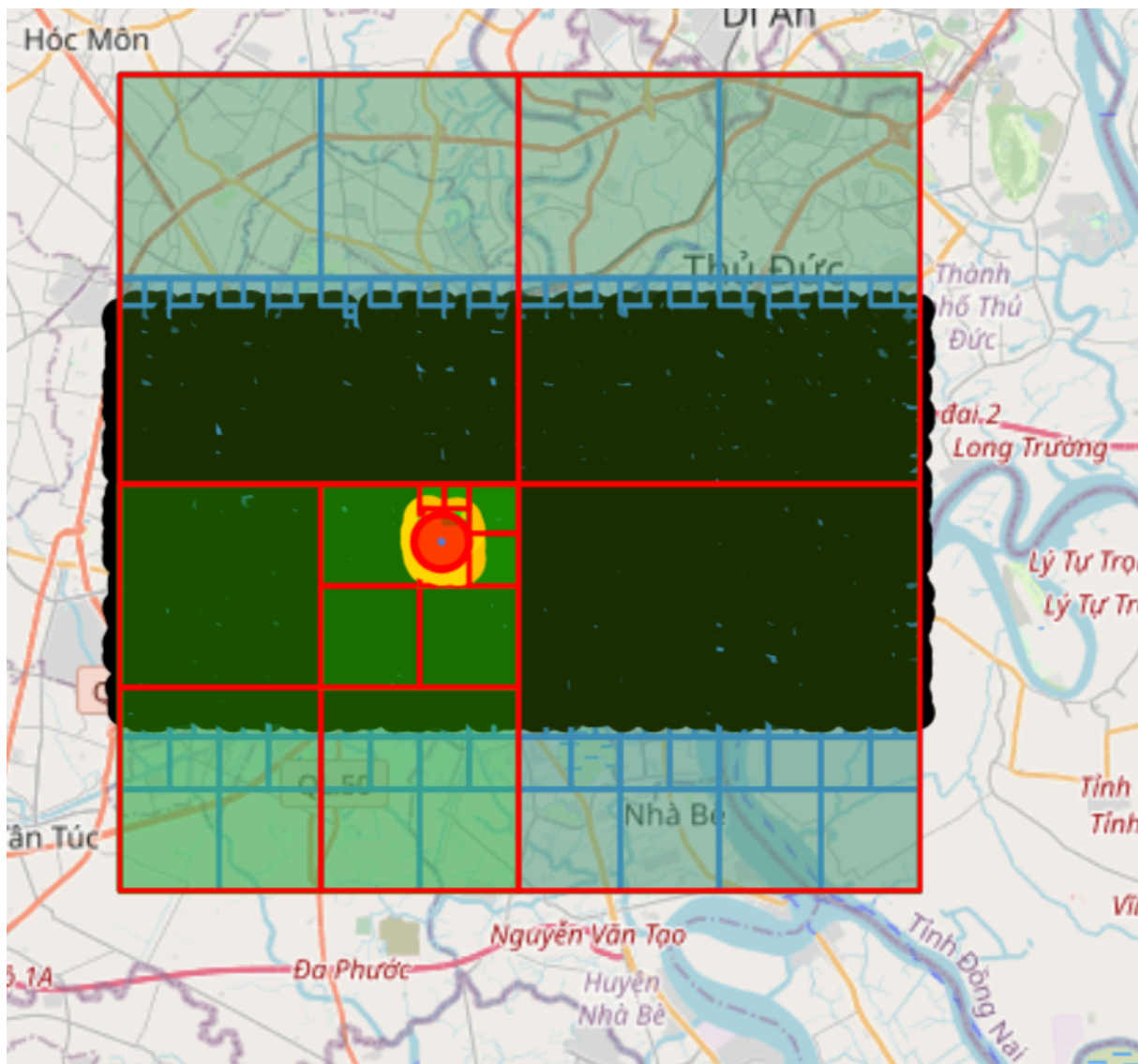
```
let searchBounds = GQTBounds(minX: -1.5, minY: -1.5, maxX: 1.5, maxY: 1.5)
for item in tree.search(with: searchBounds) as! [QuadTreeItem] {
```

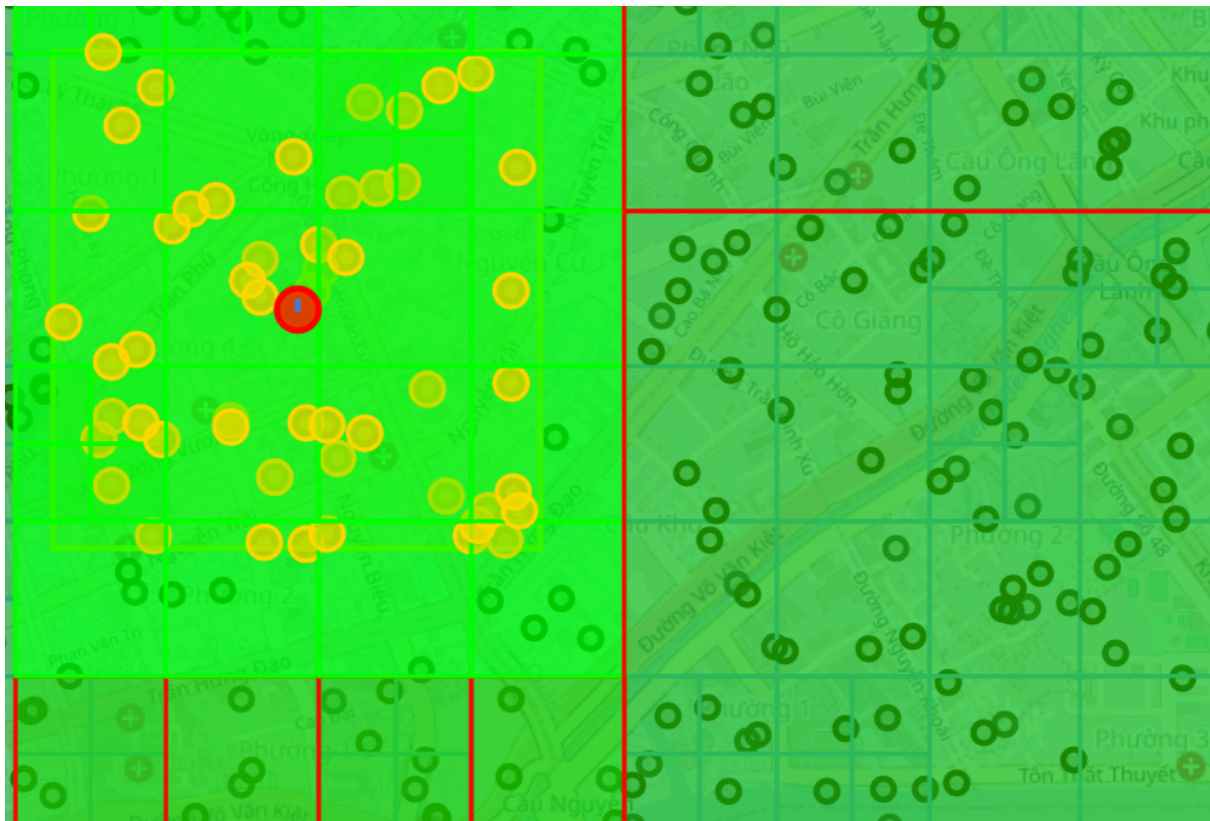
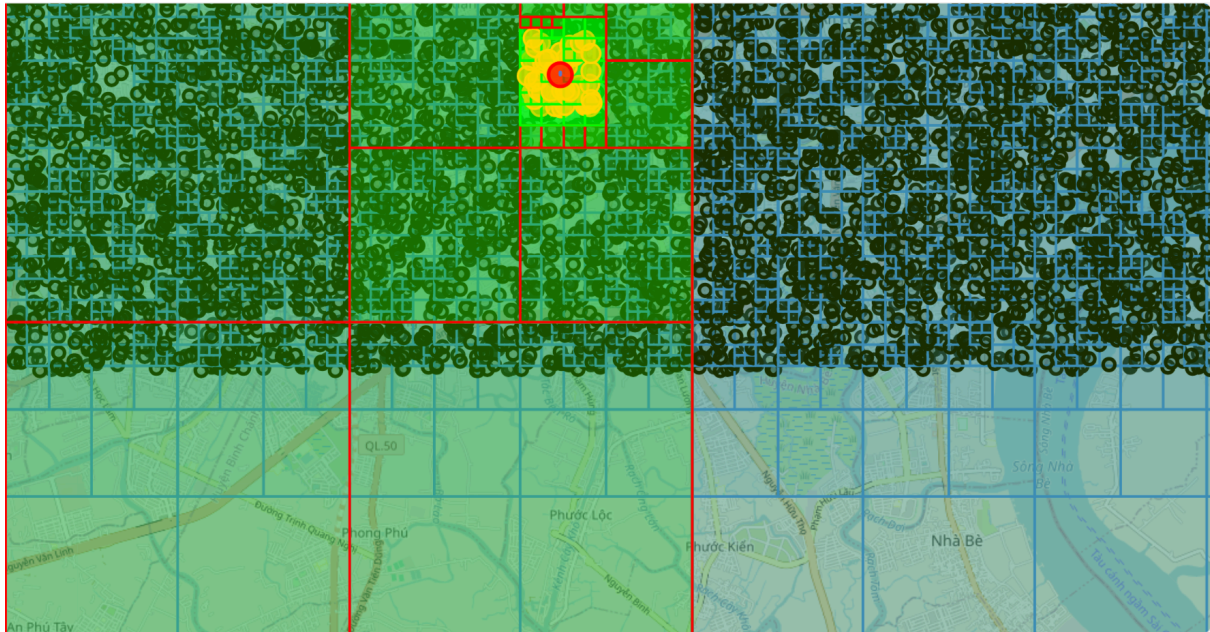
```
print("\\(item.point().x), \\(item.point().y))")
}
```

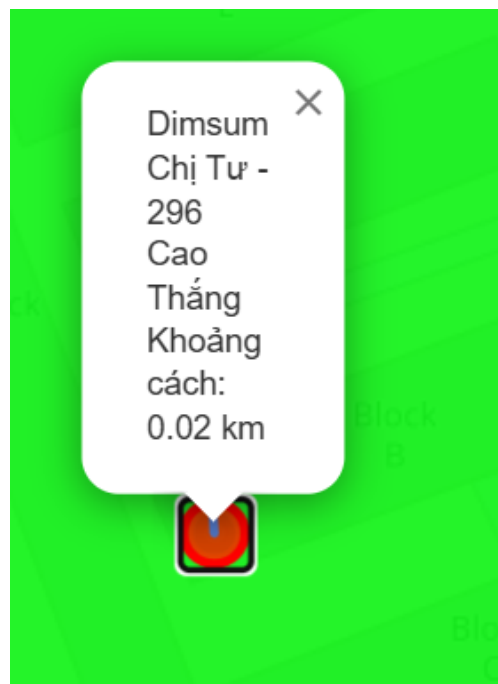
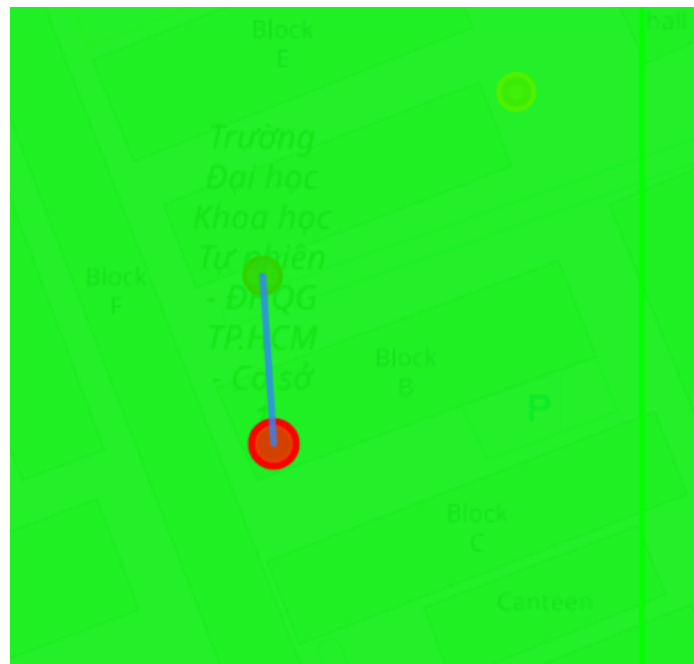
- **searchBounds:** A rectangular bounding box used to query points.
- **tree.search(with:):** A method that returns all points within the search region.

Finding Nearby Restaurants Near the University of Science (Ho Chi Minh City)

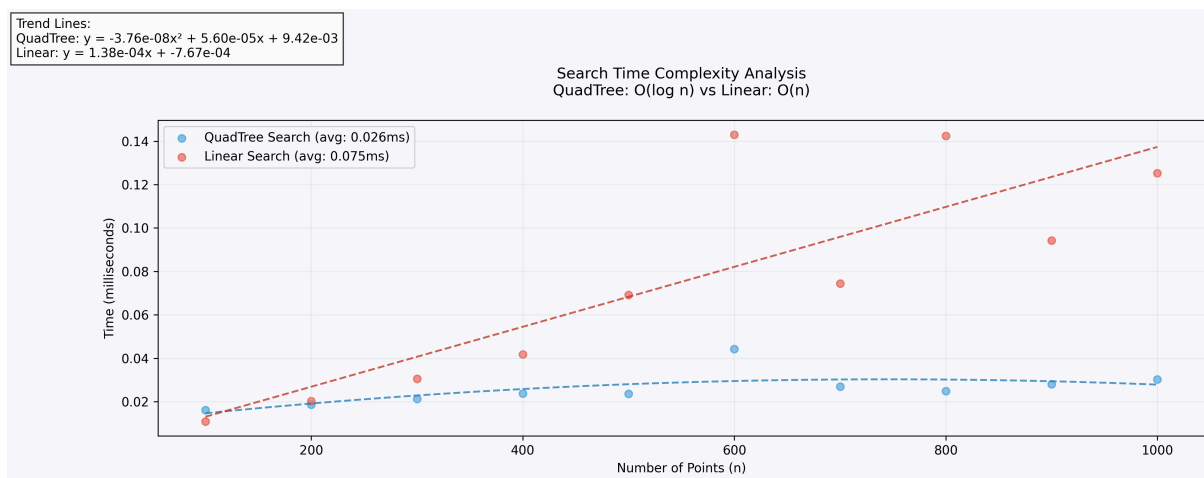
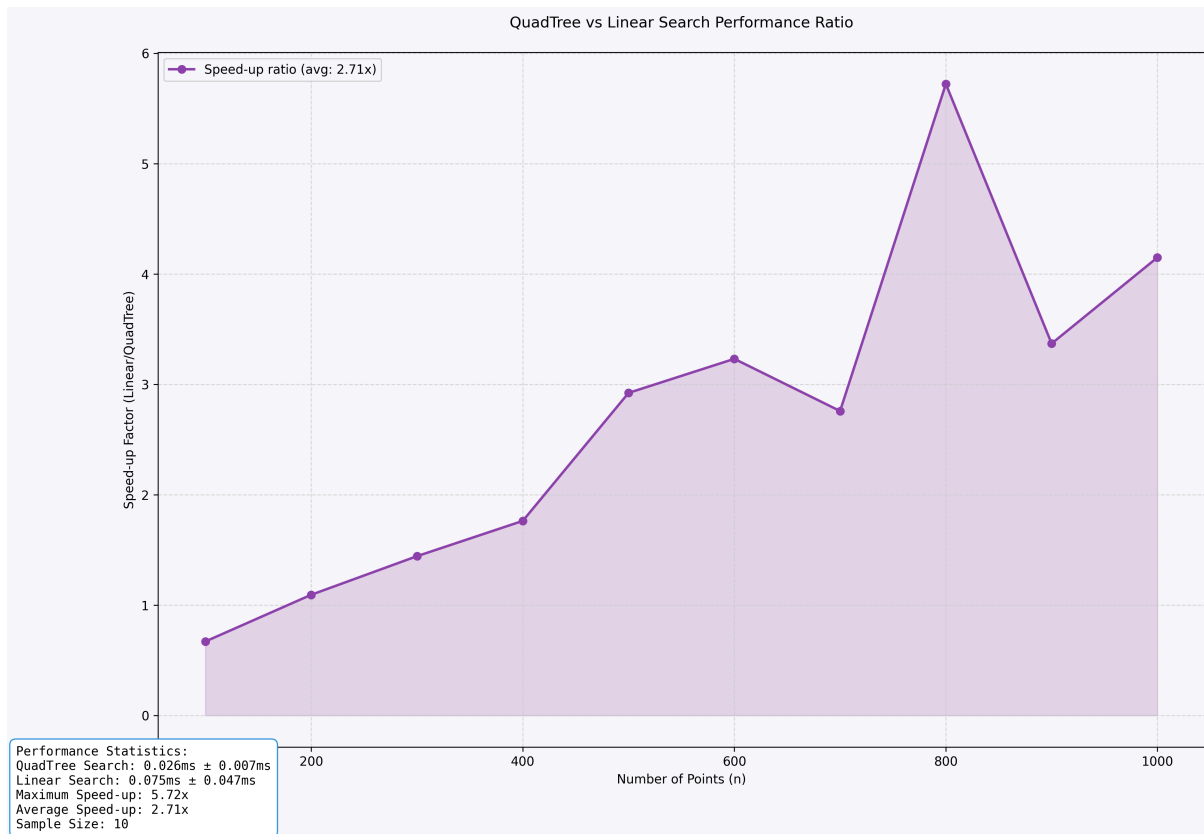
I developed a visualization program to demonstrate quadtree search operations. I inserted approximately 1,000 random restaurants across Ho Chi Minh City and then searched for the closest restaurant to the University of Science. The results are as follows:







Below is a time complexity comparison between **Linear Search** and **Quadtrees Search**:



The entire map can be viewed using the files **quadtree_search.html** and **quadtree_analysis.html**.

Additional Applications:

Beyond spatial searches, quadtrees have various other applications, including:

- **Image Union/Intersection**
- **Connected Component Labeling**
- **Mesh Generation Using Quadtrees**

For further exploration, refer to the [Wikipedia page on Quadtree](#).

7. Program assignment

Example 1

<https://leetcode.com/problems/construct-quad-tree/description/>

Problem

Given a $n * n$ matrix `grid` of `0's` and `1's` only. We want to represent `grid` with a Quad-Tree.

Return *the root of the Quad-Tree representing* `grid`.

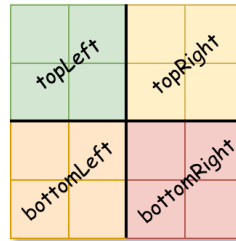
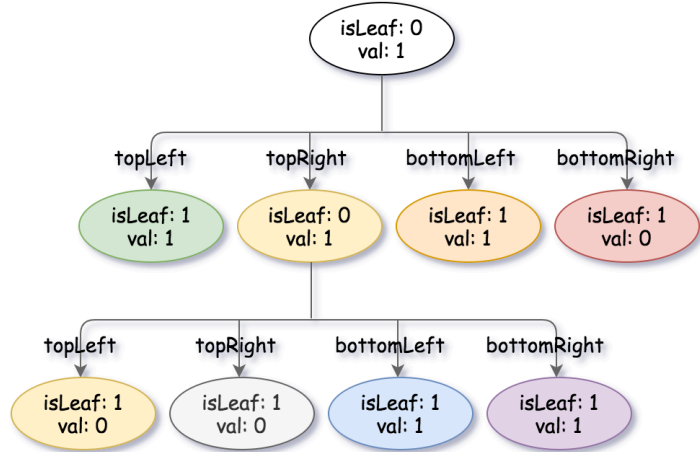
A Quad-Tree is a tree data structure in which each internal node has exactly four children. Besides, each node has two attributes:

- `val` : True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the `val` to True or False when `isLeaf` is False, and both are accepted in the answer.
- `isLeaf` : True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {  
    public boolean val;  
    public boolean isLeaf;  
    public Node topLeft;  
    public Node topRight;  
    public Node bottomLeft;  
    public Node bottomRight;  
}
```

1. If the current grid has the same value (i.e all `1's` or all `0's`) set `isLeaf` True and set `val` to the value of the grid and set the four children to Null and stop.
2. If the current grid has different values, set `isLeaf` to False and set `val` to any value and divide the current grid into four sub-grids as shown in the photo.
3. Recurse for each of the children with the proper sub-grid.

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0



Analysis

1. Subproblem: Checking Uniformity in a Rectangle

Given an $N \times M$ matrix where each cell contains either 0 or 1, determine whether any arbitrary sub-rectangle consists entirely of the same value (either all 0s or all 1s).

Naïve Approach:

Iterate through all cells in the rectangle to check if they are identical. This takes $O(n^2)$ time, which is inefficient.

Optimized Approach:

Since the matrix contains only 0s and 1s, we can use **prefix sum** to efficiently compute the sum of any sub-rectangle. Let the rectangle have dimensions $H \times W$:

- If the sum of all cells is **0**, the rectangle consists entirely of 0s.
- If the sum equals $H \times W$, the rectangle consists entirely of 1s.

Define $dp[i][j]$ as the sum of all values in the submatrix from $(0, 0)$ to (i, j) :

$$dp[i][j] = dp[i-1][j] + dp[i][j-1] - dp[i-1][j-1] + a[i][j]$$

Using this, the sum of a sub-rectangle with top-left corner (x_1, y_1) and bottom-right corner (x_2, y_2) is computed in $O(1)$:

$$sum = dp[x_2][y_2] - dp[x_1 - 1][y_2] - dp[x_2][y_1 - 1] + dp[x_1 - 1][y_1 - 1]$$

2. Applying to Quad Tree

We recursively divide the matrix (initially covering $(0, 0)$ to $(n - 1, m - 1)$) into four equal submatrices:

- If a submatrix contains only 0s or only 1s, mark it as a leaf node with `isLeaf = true` and `val = 0` or `1`, then stop recursion.
- Otherwise, split it into four smaller submatrices, which become child nodes, and recurse on each.

Setting

```
// Definition for a QuadTree node.
```

```
class Node {
```

```
public:
```

```
    bool val;
```

```
    bool isLeaf;
```

```
    Node* topLeft;
```

```
    Node* topRight;
```

```
    Node* bottomLeft;
```

```
    Node* bottomRight;
```

```
    Node() {
```

```
        val = false;
```

```
        isLeaf = false;
```

```
        topLeft = NULL;
```

```
        topRight = NULL;
```

```
        bottomLeft = NULL;
```

```
        bottomRight = NULL;
```

```
    }
```

```
    Node(bool _val, bool _isLeaf) {
```

```
        val = _val;
```

```
        isLeaf = _isLeaf;
```

```
        topLeft = NULL;
```

```
        topRight = NULL;
```

```
        bottomLeft = NULL;
```

```
        bottomRight = NULL;
```

```

    }

    Node(bool _val, bool _isLeaf, Node* _topLeft, Node* _topRight, Node* _bottomLeft, Node* _bottomRight) {
        val = _val;
        isLeaf = _isLeaf;
        topLeft = _topLeft;
        topRight = _topRight;
        bottomLeft = _bottomLeft;
        bottomRight = _bottomRight;
    }
};

class Solution {
public:
    struct point
    {
        int x;
        int y;
        point()
        {
            x=0;
            y=0;
        }
        point(int _x,int _y)
        {
            x=_x;
            y=_y;
        }
    };
    int dp[100][100];
    int sum(point topleft,point botright)
    {
        int x1=topleft.x;
        int y1=topleft.y;
        int x2=botright.x;
        int y2=botright.y;
        return dp[x2][y2]-((x1>0)?dp[x1-1][y2]:0)-((y1>0)?dp[x2][y1-1]:0)+((x1

```

```

>0&&y1>0)?dp[x1-1][y1-1]:0);
}
Node*solve(vector<vector<int>>& grid,point topleft,point botright)
{
    if(topleft.x==botright.x&&topleft.y==botright.y)
    {
        Node*res=new Node(grid[topleft.x][topleft.y],1);
        return res;
    }
    int total=sum(topleft,botright);
    if(total==0)
    {
        Node*res=new Node(0,1);
        return res;
    }
    if(total==(botright.x-topleft.x+1)*(botright.y-topleft.y+1))
    {
        Node*res=new Node(1,1);
        return res;
    }
    Node*res=new Node();
    res->val=1;
    res->isLeaf=0;
    int midpointx=(topleft.x+botright.x)/2;
    int midpointy=(topleft.y+botright.y)/2;
    res->topLeft=solve(grid,topleft,point(midpointx,midpointy));
    res->topRight=solve(grid,point(topleft.x,midpointy+1),point(midpointx,
botright.y));
    res->bottomLeft=solve(grid,point(midpointx+1,topleft.y),point(botright.
x,midpointy));
    res->bottomRight=solve(grid,point(midpointx+1,midpointy+1),botright);
    return res;
}
Node* construct(vector<vector<int>>& grid) {
    int n=grid[0].size();
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)

```

```

    {
        dp[i][j]=grid[i][j];
        if(i-1>=0)dp[i][j]+=dp[i-1][j];
        if(j-1>=0)dp[i][j]+=dp[i][j-1];
        if(i-1>=0&& j-1>=0)dp[i][j]-=dp[i-1][j-1];
    }
}
return solve(grid,point(0,0),point(n-1,n-1));
}
};

```

Example 2

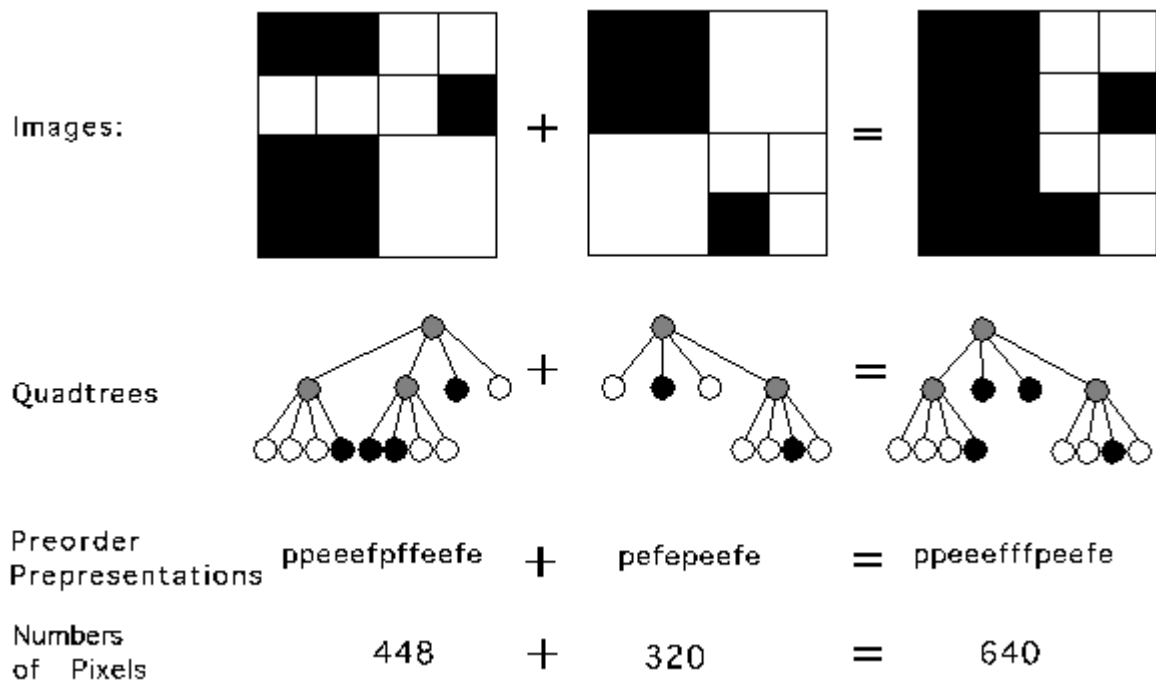
<https://dmoj.ca/problem/cc096p3>

Problem

A computer artist works with black-and-white images of size 32×32 (1024 pixels). They can "add" two images by keeping a pixel black if it appears as black in at least one of the component images.

A quadtree is a method of encoding images by recursively dividing them into four regions. If an entire region is a single colour, it is represented by a single node ('e' for white, 'f' for black). Otherwise, it is split into four quadrants and represented in pre-order with 'p' (parent) followed by the four subregions in a fixed order.

Your task is to compute the number of black pixels in the resulting image after adding two images given their quadtree representations.



Input:

- An integer (N) representing the number of test cases.
- Each test case consists of two strings, each representing a quadtree in pre-order notation.

Output:

- For each test case, print There are X black pixels. where X is the number of black pixels in the final image.

Sample input

```
3
ppeeeffpffeefe
pefepeeefe
peeef
peeefe
peeef
peepefe
```

Sample output

```
There are 640 black pixels.
There are 512 black pixels.
```

There are 384 black pixels.

Setting

```
#include <bits/stdc++.h>
using namespace std;
struct quadtree
{
    int sum;
    int type;
    int val;
    quadtree *nw,*ne,*sw,*se;
    quadtree(int sum,int type,int val)
    {
        this->sum=sum;
        this->type=type;
        this->val=val;
        nw=ne=sw=se=NULL;
    }
};
void insert(quadtree *&root,const string &s,int& id)
{
    if(id>=s.size())
    {
        return;
    }
    if(root->type==1)return;
    if(s[id]!='p')
    {
        if(root->nw==NULL)
        {
            root->nw=new quadtree(0,0,root->val/4);
            root->ne=new quadtree(0,0,root->val/4);
            root->sw=new quadtree(0,0,root->val/4);
            root->se=new quadtree(0,0,root->val/4);
        }
        insert(root->nw,s,++id);
        insert(root->ne,s,++id);
```

```

        insert(root->sw,s,++id);
        insert(root->se,s,++id);
        root->sum=root->nw->sum+root->ne->sum+root->sw->sum+root->se-
>sum;
    }
    else if(s[id]=='f')
    {
        root->type=1;
        root->sum=root->val;
        return;
    }
    return;
}
int main()
{
    //cout<<"how many test cases\n";
    int test;
    cin>>test;
    while(test--)
    {
        string s1;
        cin>>s1;
        string s2;
        cin>>s2;
        quadtree *root=new quadtree(0,0,1024);
        int id=0;
        insert(root,s1,id);
        id=0;
        insert(root,s2,id);
        cout<<"There are "<<root->sum<<" black pixels."<<endl;
        //There are 640 black pixels.
    }
}

```

8.Conclusion:

In conclusion, the Quadtree data structure provides an efficient method for spatial data partitioning, particularly in applications requiring hierarchical organization. Its adaptability to

various domains, such as computer graphics, geographical information systems, and collision detection in games, highlights its versatility. By recursively subdividing space, Quadtree optimizes storage and query operations, significantly improving computational efficiency. However, its performance depends on factors such as data distribution and depth constraints, requiring careful implementation. Future research can explore hybrid approaches integrating Quadtrees with other spatial indexing techniques to further enhance performance and scalability.

References:

- [1] Samet, H. (1984). The Quadtree and Related Hierarchical Data Structures. *ACM Computing Surveys (CSUR)*, 16(2), 187-260.
- [2] Finkel, R. A., & Bentley, J. L. (1974). Quad Trees: A Data Structure for Retrieval on Composite Keys. *Acta Informatica*, 4, 1-9.
- [3] De Berg, M., Van Kreveld, M., Overmars, M., & Schwarzkopf, O. (2008). *Computational Geometry: Algorithms and Applications*. Springer.
- [4] Beckmann, N., Kriegel, H. P., Schneider, R., & Seeger, B. (1990). The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. *ACM SIGMOD Record*, 19(2), 322-331.
- [5] Agarwal, P. K., & Erickson, J. (1999). Geometric Range Searching and Its Relatives. *Contemporary Mathematics*, 223, 1-56.